Лабораторная работа 1

Почему глубокая сеть обучается или не обучается

Инициализация, нормализация и остаточные связи

Курс «Нейронные сети»

Редакция 4 от 8 октября 2026 г.

1 Введение

Чем глубже сеть, тем труднее её обучить. Каждый слой немного меняет масштаб сигнала. Если слой уменьшает его в 0,9 раза, после 50 слоёв останется $0{,}9^{50} \approx 0{,}005$ исходной величины. Если увеличивает в 1,1 раза, сигнал вырастет примерно в 117 раз. С градиентами при обратном проходе происходит то же самое: они затухают или «взрываются».

Способность глубокой сети сохранять сигнал и градиенты в рабочем диапазоне и при этом успешно обучаться называют *обучаемостью* (trainability). В этой работе выясняется, какие решения делают глубокую сеть обучаемой: способ инициализации весов, нормализация и остаточные связи.

Важно различать два явления:

1. свойства сети **в момент инициализации**, до первого шага оптимизатора;
2. свойства **процесса обучения**, которые возникают из взаимодействия архитектуры, оптимизатора, скорости обучения и данных.

Первое можно измерить за секунды, без обучения. Второе требует запусков обучения. Работа показывает, насколько первое предсказывает второе.

Все эксперименты выполняются в проекте из ЛР0: у каждого запуска есть `EXP_ID`, `RUN_ID`, паспорт и итоговая конфигурация. Числа в отчёте сопровождаются ссылками на `RUN_ID`.

2 Цель работы

Цель работы — экспериментально установить, как глубина, инициализация, нормализация и остаточные связи влияют на обучаемость глубокой сети, и построить **карту обучаемости**. Карта связывает результат обучения с измеримыми свойствами активаций и градиентов.

Какие свойства архитектуры и инициализации определяют, будет ли глубокая сеть обучаемой, и можно ли увидеть это до начала обучения?

3 Что нужно знать заранее

Если какой-то пункт незнаком, восполните пробел по указанному источнику.

- Обратное распространение ошибки и правило дифференцирования сложной функции — [1, разд. 2.5], [2, разд. 6.5].
- Функции активации ReLU, sigmoid, tanh — [1, разд. 3.3], [2, разд. 6.3].
- SGD с импульсом и Adam — [1, разд. 4.4, 4.5], [2, разд. 8.3, 8.5].
- Инициализация весов — [1, разд. 4.2], [2, разд. 8.4].
- Нормализация по мини-батчам (BatchNorm) — [1, разд. 4.3], [2, разд. 8.7.1].
- Остаточные сети (ResNet) — [1, разд. 5.4], [12].
- Хуки PyTorch для перехвата промежуточных значений — документация `torch.nn.Module` [4].
- Проект для экспериментов из ЛР0.

4 Результаты обучения

После выполнения работы вы сможете:

1. отличать затухание сигнала при прямом проходе от затухания градиента при обратном;
2. объяснять, как ширина слоя, функция активации и масштаб инициализации определяют рост или затухание сигнала;
3. измерять статистики активаций и градиентов по слоям;
4. распознавать затухающие и взрывающиеся градиенты, насыщение и «мёртвые» нейроны;
5. объяснять, как остаточная связь помогает сигналу и градиенту и зачем уменьшать масштаб остаточной ветви;
6. делать выводы по нескольким seeds, а не по одному запуску.

5 Модели и обозначения

5.1 Многослойная сеть

Базовая модель — полносвязная сеть (многослойный перцептрон). Слой с номером l берёт выход предыдущего слоя, умножает его на матрицу весов, прибавляет смещение, нормализует и применяет функцию активации:

$$z^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}, \qquad h^{(l)} = \phi\big(N_l(z^{(l)})\big), \qquad l = 1, \dots, L, \tag{1.1}$$

где

- $h^{(0)} = x$ — вход сети, $h^{(l)} \in \mathbb{R}^{n_l}$ — выход слоя l, n_l — ширина слоя (число нейронов);
- $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ и $b^{(l)}$ — веса и смещения;
- $z^{(l)}$ — значения до функции активации (pre-activation);
- N_l — нормализация: её нет, BatchNorm или LayerNorm (раздел 5.4);
- ϕ — функция активации, применяемая к каждому элементу;
- L — глубина, число скрытых слоёв.

Порядок «нормализация, затем активация» фиксируется в конфигурации и одинаков для всех сравниваемых сетей.

В работе используются две функции активации:

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}. \tag{1.2}$$

ReLU обнуляет отрицательные значения, поэтому долю нулей на её выходе можно измерить. Функция tanh ограничена интервалом $(-1, 1)$; при больших $|z|$ её производная близка к нулю. Это называют *насыщением*: через насыщенный нейрон градиент почти не проходит.

5.2 Как масштаб сигнала меняется от слоя к слою

«Амплитуду» сигнала на выходе слоя будем измерять средним квадратом его значений:

$$q^{(l)} = \frac{1}{n_l} \sum_{j=1}^{n_l} \mathbb{E}\big[(h_j^{(l)})^2\big], \tag{1.3}$$

где $\mathbb{E}$ — математическое ожидание по входам и случайным весам.

Сделаем предположения, как в работах [8, 9]: веса слоя независимы, имеют нулевое среднее и дисперсию σ_l^2 и не зависят от входа слоя, а смещения равны нулю. Тогда для ReLU-сети без нормализации каждый слой умножает средний квадрат на одно и то же число:

$$q^{(l)} = \chi_l \, q^{(l-1)}, \qquad \chi_l = \frac{n_{l-1}\,\sigma_l^2}{2}, \qquad \text{откуда} \qquad q^{(l)} = \chi^l \, q^{(0)} \tag{1.4}$$

для слоёв одинаковой ширины с общим множителем χ. Множитель $1/2$ появляется потому, что значения $z^{(l)}$ симметричны относительно нуля, а ReLU обнуляет их отрицательную половину. Если $\chi < 1$, сигнал затухает экспоненциально с глубиной; если $\chi > 1$ — экспоненциально растёт. Масштаб сохраняется только при $\chi = 1$.

Отсюда три схемы инициализации, которые сравниваются в работе.

Инициализация He (Kaiming) [9] выбирает $\chi = 1$:

$$W_{ji}^{(l)} \sim \mathcal{N}\left(0,\ \frac{2}{n_{l-1}}\right). \tag{1.5}$$

Инициализация Xavier (Glorot) [8] выведена для функций, почти линейных около нуля, например tanh:

$$W_{ji}^{(l)} \sim \mathcal{N}\left(0,\ \frac{2}{n_{l-1} + n_l}\right). \tag{1.6}$$

Для слоёв одинаковой ширины n она даёт $\sigma^2 = 1/n$. В ReLU-сети это означает $\chi = 1/2$: сигнал ослабевает вдвое на каждом слое.

Гауссовская инициализация с постоянным масштабом — намеренно неудачная схема, где σ не зависит от ширины:

$$W_{ji}^{(l)} \sim \mathcal{N}(0, \sigma^2), \qquad \chi = \frac{n\,\sigma^2}{2}. \tag{1.7}$$

Например, при $n = 256$ и $\sigma = 0{,}05$ получаем $\chi = 0{,}32$. После 10 слоёв сигнал уменьшится в $0{,}32^{10} \approx 10^{-5}$ раз.

Для tanh точная формула сложнее. При малых сигналах tanh почти линейна, и множитель близок к $n\sigma^2$; при больших она насыщается.

Подробнее: [1, разд. 4.2]; [2, разд. 8.4]; [8, 9]; функции инициализации PyTorch — [3].

5.3 Градиент при обратном проходе

При обратном проходе градиент идёт через слои в обратном порядке. На каждом слое он умножается на матрицу частных производных выхода слоя по его входу — *якобиан* слоя:

$$\delta^{(l)} = \delta^{(L)}\, J_L\, J_{L-1} \cdots J_{l+1}, \qquad J_k = \frac{\partial h^{(k)}}{\partial h^{(k-1)}} = D_k W^{(k)}, \tag{1.8}$$

где $\delta^{(l)} = \partial \mathcal{L} / \partial h^{(l)}$ — градиент функции потерь $\mathcal{L}$ по выходу слоя l, записанный строкой; D_k — диагональная матрица из производных $\phi'(z_j^{(k)})$. Поэтому с градиентом происходит то же, что с сигналом. При тех же предположениях для ReLU средний квадрат градиента на каждом слое умножается на $n_l \sigma_l^2 / 2$. Для слоёв одинаковой ширины это тот же множитель χ, но градиент затухает в обратную сторону — от выхода к входу [8].

Подробнее: [2, разд. 6.5, 8.2].

5.4 Нормализация

Нормализация принудительно приводит значения к нулевому среднему и единичному разбросу. Поэтому она частично исправляет неудачный масштаб весов.

BatchNorm [10] нормирует каждый нейрон j по объектам batch:

$$\mu_j = \frac{1}{B}\sum_{b=1}^{B} z_{b,j}, \quad s_j^2 = \frac{1}{B}\sum_{b=1}^{B}(z_{b,j} - \mu_j)^2, \quad \mathrm{BN}(z)_{b,j} = \gamma_j \frac{z_{b,j} - \mu_j}{\sqrt{s_j^2 + \varepsilon_{\mathrm{BN}}}} + \beta_j, \tag{1.9}$$

где B — размер batch, $z_{b,j}$ — значение нейрона j на объекте b, γ_j и β_j — обучаемые масштаб и сдвиг, $\varepsilon_{\mathrm{BN}}$ — малая константа (в PyTorch по умолчанию 10^{-5}). При обучении BatchNorm использует статистики текущего batch, а при оценке — средние, накопленные за обучение. Поэтому её выход зависит от режима (`train` или `eval`) и от состава batch. При $B = 1$ в режиме обучения $s_j^2 = 0$, и нормализация теряет смысл.

LayerNorm [11] нормирует каждый объект b по нейронам слоя:

$$\mu_b = \frac{1}{n}\sum_{j=1}^{n} z_{b,j}, \quad s_b^2 = \frac{1}{n}\sum_{j=1}^{n}(z_{b,j} - \mu_b)^2, \quad \mathrm{LN}(z)_{b,j} = \gamma_j \frac{z_{b,j} - \mu_b}{\sqrt{s_b^2 + \varepsilon_{\mathrm{LN}}}} + \beta_j. \tag{1.10}$$

LayerNorm не зависит от состава batch и одинаково работает в обоих режимах.

Подробнее: [1, разд. 4.3]; [2, разд. 8.7.1]; [5, 6].

5.5 Остаточный блок

Остаточный блок (residual block) прибавляет к результату вычислений исходный вход. У сигнала и градиента появляется обходной путь:

$$h^{(l+1)} = h^{(l)} + \alpha\, F_l\big(h^{(l)}\big), \qquad F_l(h) = W_{l,2}\,\phi\big(N_l(W_{l,1}h + b_{l,1})\big) + b_{l,2}, \tag{1.11}$$

где F_l — остаточная ветвь из двух линейных слоёв ширины n, $\alpha > 0$ — масштаб ветви. В сети из L блоков $2L$ линейных слоёв; это учитывается при сравнении глубины с обычной сетью.

Якобиан блока

$$J_l^{\mathrm{res}} = I + \alpha \frac{\partial F_l}{\partial h} \tag{1.12}$$

содержит единичную матрицу I: градиент проходит через блок напрямую, минуя веса и активации ветви [12].

У обходного пути есть цена. При тех же предположениях о весах средний квадрат сигнала растёт с каждым блоком:

$$q^{(l+1)} = q^{(l)}\big(1 + \alpha^2 c\big), \qquad \text{откуда} \qquad q^{(L)} = q^{(0)}\big(1 + \alpha^2 c\big)^L, \tag{1.13}$$

где c — во сколько раз ветвь меняет средний квадрат своего входа. При инициализации Не обоих слоёв ветви и без нормализации $c = 2$. При $\alpha = 1$ рост экспоненциален: при $c = 2$ и $L = 16$ средний квадрат вырастет в $3^{16} \approx 4 \cdot 10^7$ раз. При $\alpha = 1/\sqrt{L}$ множитель $(1 + c/L)^L$ не превышает e^c при любой глубине [13]. Поэтому в работе сравниваются оба режима.

5.6 Как измерять обучаемость

Итоговой точности для вывода об обучаемости недостаточно. Для каждого запуска измеряются четыре величины.

Уровень ошибки после обучения. Модель, которая угадывает класс случайно, имеет ошибку (cross-entropy) $\ln K$, где K — число классов; для $K = 10$ это примерно 2,3. Сравним с ним ошибку после T шагов обучения:

$$\rho_T = \frac{\mathcal{L}_T}{\ln K}, \tag{1.14}$$

где $\mathcal{L}_T$ — ошибка на фиксированном подмножестве обучающих данных после T шагов. Сеть считается *обучаемой*, если $\rho_T \le 0{,}5$, и *необучаемой*, если $\rho_T \ge 0{,}9$; промежуточные значения — пограничный режим.

Размер градиента в слое. Чтобы сравнивать слои разного размера, норму градиента делят на корень из числа параметров:

$$g_l = \frac{\big\|\nabla_{\theta_l}\mathcal{L}\big\|_2}{\sqrt{|\theta_l|}}, \tag{1.15}$$

где θ_l — параметры слоя l, $|\theta_l|$ — их число. Во время обучения берётся медиана g_l по измеряемым шагам.

Дисбаланс градиентов между слоями:

$$\mathrm{GI} = \log_{10} \frac{\max_l g_l + \varepsilon}{\min_l g_l + \varepsilon}. \tag{1.16}$$

Значение $\mathrm{GI} = 1$ означает, что градиенты слоёв различаются в 10 раз, $\mathrm{GI} = 3$ — в 1000 раз.

Изменение разброса активаций с глубиной:

$$\mathrm{AVD} = \max_l \big|\log_{10}(v_l + \varepsilon) - \log_{10}(v_1 + \varepsilon)\big|, \tag{1.17}$$

где v_l — дисперсия активаций слоя l на пробном batch, $\varepsilon = 10^{-12}$ — константа для защиты от логарифма нуля.

Подробнее: [2, разд. 8.2].

5.7 Сводка обозначений

Обозначение	Смысл
L, n_l, n	глубина, ширина слоя l, общая ширина
$W^{(l)}$, $b^{(l)}$, σ_l^2	веса, смещения, дисперсия весов слоя
$z^{(l)}$, $h^{(l)}$, ϕ, N_l	значения до и после активации, функция активации, нормализация
$q^{(l)}$, χ	средний квадрат выхода слоя, множитель на слой
$\delta^{(l)}$, J_k, D_k	градиент по выходу слоя, якобиан слоя, производные активации
B, γ_j, β_j	размер batch, параметры нормализации
F_l, α, c	остаточная ветвь, её масштаб, её коэффициент передачи
ρ_T, g_l, GI, AVD, ε	показатели обучаемости, малая константа

6 Данные и оборудование

Основной набор данных — FashionMNIST [16]: 70 000 изображений одежды 28×28 пикселей, 10 классов. Обучение на нём быстрое и позволяет сделать десятки запусков. Разбиение на `train`, `val` и `test` создаётся механизмом ЛР0 и одинаково для всех запусков. Допускается CIFAR-10.

Оборудование. Видеокарта не нужна. Эпоха обучения эталонной сети (обычная сеть, $L = 32$, ширина 256) занимает на процессоре десятки секунд. Если это слишком медленно, уменьшите ширину до 128, сохранив остальные условия.

7 Индивидуальный профиль

Начиная с ЛР1, параметры каждой работы вычисляются из отдельного хеша: номера зачётки вместе с номером работы. Поэтому профили разных работ независимы. Добавьте в генератор профиля из ЛР0 функцию `lab_pick` и три поля ЛР1:

```
def lab_pick(student_id: str, lab: int, a: int, b: int, options: list):
    hk = hashlib.sha256(f"{student_id}:lab{lab}".encode("utf-8")).hexdigest()
    return options[int(hk[a:b], 16) % len(options)]

profile["lab1_depths"] = lab_pick(sid, 1, 0, 2, [[4, 16, 32, 64], [8, 24, 48, 96]])

profile["lab1_bad_sigma"] = lab_pick(sid, 1, 2, 4, [0.02, 0.05, 0.10])
profile["lab1_optimizer"] = lab_pick(sid, 1, 4, 6, ["sgd_momentum", "adamw"])
```

Здесь `sid` — номер зачётки, `profile` — словарь профиля из ЛР0. Профиль задаёт набор глубин, масштаб σ неудачной гауссовской инициализации (1.7) и оптимизатор для обучения. Заметьте: при ширине 256 значения $\sigma = 0{,}02$ и $0{,}05$ дают затухание сигнала, а $\sigma = 0{,}10$ — рост ($\chi = 1{,}28$).

8 Задания

8.1 Задание 1. Монитор активаций и градиентов

Что сделать.

1. Реализуйте компонент, который для каждого выбранного слоя записывает: среднее, стандартное отклонение и дисперсию v_l, средний квадрат $q^{(l)}$ (1.3), минимум и максимум, долю нулевых значений (для ReLU) и долю насыщенных значений, у которых $|\phi'(z)| < 0{,}1$ (для tanh).
2. Для каждого слоя с параметрами записывайте норму градиента, величину g_l (1.15) и отношение нормы градиента к норме параметров слоя. Отдельно записывайте средний квадрат градиента по выходу слоя $\delta^{(l)}$ (1.8).
3. Используйте хуки (`register_forward_hook`, `register_full_backward_hook`) или хуки тензоров, чтобы не менять метод `forward()` модели [4].
4. Считайте статистики в `float64`: при сильном росте сигнала квадраты значений не помещаются в `float32`.

Что должно получиться. Для любой модели одним вызовом строится таблица «слой → статистики».

Как проверить себя. С включённым и выключенным монитором выход модели и градиенты совпадают (тесты 1 и 2).

8.2 Задание 2. Конфигурируемая глубокая сеть

Что сделать.

1. Реализуйте генератор обычных сетей (1.1) и остаточных сетей (1.11). Всё задаётся конфигурацией: глубина L, ширина n, функция активации, нормализация и её место, схема инициализации (`he`, `xavier`, `gaussian` с заданным σ), масштаб ветви α (1 или $1/\sqrt{L}$).
2. Инициализируйте веса явно по конфигурации, а смещения — нулями. Не полагайтесь на инициализацию PyTorch по умолчанию: в этой работе инициализация — исследуемый фактор [3].
3. Для измерений до обучения нужна функция потерь без меток. Используйте линейную пробную функцию

$$\mathcal{L}_{\text{probe}} = \frac{1}{B}\sum_{b=1}^{B} u^{\top} h_b^{(L)}, \qquad \|u\|_2 = 1, \tag{1.18}$$

   где $h_b^{(L)}$ — выход последнего скрытого слоя на объекте b, а u — фиксированный случайный вектор единичной длины, порождённый из seed профиля. Её градиент по выходу сети одинаков для всех конфигураций, поэтому изменение градиента от слоя к слою отражает только свойства сети.
4. Пробный batch — фиксированные реальные изображения, $B \geq 64$. Размер batch и режим (`train` или `eval`) записывайте в паспорт запуска.

Что должно получиться. Любая сеть из экспериментов создаётся изменением конфигурации, без правки кода.

Как проверить себя. Выборочная дисперсия весов совпадает с целевой (тест 3); остаточный блок с нулевой ветвью ничего не меняет (тест 4).

8.3 Задание 3. Сводка обучаемости

Что сделать.

1. Для каждого запуска автоматически вычисляйте ρ_T (1.14), GI (1.16), AVD (1.17), наибольшие доли нулевых и насыщенных значений и флаг численного сбоя — появления NaN или Inf в ошибке, активациях или градиентах.
2. Сохраняйте сводку в `summary.json` запуска, например:

```
"trainability": {"rho_T": 0.21, "GI": 0.8, "AVD": 0.4,
                 "max_zero_fraction": 0.55, "numerical_failure": false}
```

3. Убедитесь, что команда `compare` из ЛР0 строит по этим сводкам общую таблицу для десятков запусков.

Что добавляется в общий проект. Модули `src/diagnostics/` (`activations.py`, `gradients.py`, `trainability.py`) и `src/models/` (`deep_mlp.py`, `residual_mlp.py`). Ими вы будете пользоваться в следующих работах для анализа любых глубоких моделей.

Как проверить себя. При искусственно внесённом NaN сводка ставит флаг численного сбоя (тест 5).

9 Эксперименты

Перед каждым экспериментом запишите в отчёт гипотезу: что вы ожидаете увидеть и почему, со ссылкой на формулу из раздела 5. Эксперименты E1 и E2 — поисковые, достаточно одного seed. Главный вывод работы проверяется в E3 на трёх seeds. Если результат разошёлся с предсказанием, это допустимо: укажите, какое предположение вывода не выполнилось и почему.

9.1 Эксперимент E1. Сигнал в сети до обучения

Вопрос. Можно ли по измерениям сети до обучения предсказать, какие конфигурации будет трудно обучить?

Что сделать. Оптимизатор не используется. Возьмите обычную ReLU-сеть ширины 256 без нормализации, все глубины из профиля и три схемы инициализации: He (1.5), Xavier (1.6) и гауссовскую с σ из профиля (1.7). Для каждой пары «глубина, инициализация» выполните прямой проход на пробном batch, вычислите пробную функцию (1.18) и выполните обратный проход. Постройте графики по слоям в логарифмическом масштабе: $q^{(l)}$, средний квадрат $\delta^{(l)}$ и g_l.

Сравните измерения с теорией через измеренный множитель на слой — среднее геометрическое изменения за слой:

$$\hat{\chi}_{\text{fwd}} = \left(\frac{q^{(L)}}{q^{(1)}}\right)^{\frac{1}{L-1}}, \qquad \hat{\chi}_{\text{bwd}} = \left(\frac{\overline{(\delta^{(1)})^2}}{\overline{(\delta^{(L)})^2}}\right)^{\frac{1}{L-1}}, \tag{1.19}$$

где черта сверху — среднее по нейронам слоя и объектам batch. Слои, где значения ушли в машинный ноль ($q^{(l)} < 10^{-30}$), в расчёт не берутся; L тогда — номер последнего учтённого слоя.

Порог приёмки. На наибольшей глубине профиля для каждой схемы инициализации $\hat{\chi}_{\text{fwd}}$ и $\hat{\chi}_{\text{bwd}}$ отличаются от предсказанного χ не более чем на 20%. Предсказание: $\chi = 1$ для He, $\chi = 1/2$ для Xavier, $\chi = n\sigma^2/2$ для гауссовской схемы. Множитель считается по скрытым слоям одинаковой ширины, начиная со второго.

Граница применимости. Повторите измерение для гауссовской схемы после обучения в течение одной эпохи. Выполняется ли формула (1.4) и теперь? Какое её предположение нарушилось первым?

9.2 Эксперимент E2. Глубина и остаточные связи

Вопрос. С какой глубины обычная сеть перестаёт обучаться и спасает ли её остаточная связь?

Что сделать. Инициализация He, ReLU, без нормализации, глубины из профиля. Сравните три архитектуры: обычную сеть, остаточную с $\alpha = 1$ и остаточную с $\alpha = 1/\sqrt{L}$.

1. До обучения измерьте $q^{(L)}/q^{(0)}$ для остаточных сетей и коэффициент c — отношение среднего квадрата выхода ветви к среднему квадрату её входа в первом блоке.
2. Обучите каждую сеть 3 эпохи оптимизатором из профиля. Скорость обучения подберите один раз для самой мелкой обычной сети по сетке $\{10^{-3}, 3\cdot10^{-3}, 10^{-2}, 3\cdot10^{-2}, 10^{-1}\}$ и используйте для всех запусков.
3. Для каждой пары «архитектура, глубина» запишите ρ_T, GI, AVD и точность на `val`. Постройте графики ρ_T и GI от глубины для трёх архитектур.

Порог приёмки.

- При $\alpha = 1$ измеренный рост за блок $\left(q^{(L)}/q^{(0)}\right)^{1/L}$ отличается от $1 + c$ не более чем на 20%.
- При $\alpha = 1/\sqrt{L}$ отношение $q^{(L)}/q^{(0)}$ не превышает $1{,}2\,e^c$ на всех глубинах.
- Для каждой архитектуры указана наименьшая глубина, на которой сеть становится необучаемой ($\rho_T \geq 0{,}9$) или даёт численный сбой. Если такой глубины в профиле нет, это явно указано.

Граница применимости. Найденная глубина для обычной сети и поведение остаточной сети с $\alpha = 1$ на самой большой глубине профиля. Совпадает ли наблюдаемое с предсказанием (1.13)?

9.3 Эксперимент E3. Основное сравнение

Вопрос. Выдерживает ли главный вывод работы смену seed?

Что сделать. По результатам E1–E2 выберите две конфигурации, A и B, между которыми формулируется главный вывод. Например: обычная сеть против остаточной с $\alpha = 1/\sqrt{L}$ на самой большой обучаемой глубине. До запусков запишите в файл `hypothesis.md` и зафиксируйте в Git:

1. чем различаются A и B — ровно один фактор;
2. что зафиксировано: остальные факторы, данные, число эпох, seeds;
3. какую разницу вы ожидаете, по какой метрике и с каким знаком;
4. какой результат опровергнет гипотезу.

Обучите каждую конфигурацию с тремя общими seeds $s = 1, 2, 3$. Для основной метрики m вычислите парные разности:

$$\Delta_s = m(\mathrm{A}, s) - m(\mathrm{B}, s), \qquad \bar{\Delta} = \frac{1}{3}\sum_{s=1}^{3} \Delta_s, \qquad s_\Delta = \sqrt{\frac{1}{2}\sum_{s=1}^{3}(\Delta_s - \bar{\Delta})^2}, \tag{1.20}$$

где $m(\cdot, s)$ — метрика конфигурации при seed s, $\bar{\Delta}$ — средняя разность, s_Δ — её стандартное отклонение.

Порог приёмки. Вывод «A и B различаются» считается установленным, если все три разности Δ_s одного знака и $|\bar{\Delta}| \geq 2s_\Delta$. Иначе вывод формулируется как «различие не установлено». В отчёте приводятся все три значения Δ_s, а не только среднее. Расхождение результата с гипотезой не снижает оценку, если оно объяснено.

Граница применимости. Три seeds описывают разброс на данном разбиении данных. Перенести вывод на другие данные они не позволяют — укажите это в разделе об ограничениях.

10 Контрфактические вопросы

Ответ опирается на ваши числа со ссылками на `RUN_ID`. Если для ответа нужен дополнительный запуск, выполните его.

1. Оцените по своим данным E1 множитель $\hat{\chi}_{\mathrm{fwd}}$ для инициализации He. Каким по формуле (1.4) будет $q^{(256)}/q^{(1)}$? Почему стабильность при $L = 16$ не гарантирует обучаемость при $L = 256$? Какое предположение вывода нарушается первым?
2. Ширину увеличили в 4 раза, сохранив гауссовскую инициализацию с тем же σ. Предскажите новый χ по формуле (1.7) и проверьте одним запуском E1.
3. В E1 при неудачной инициализации градиент по выходам слоёв $\delta^{(l)}$ затухает. Почему при этом величины g_l почти одинаковы во всех скрытых слоях, кроме первого? Найдите это на своих графиках и объясните, сопоставив формулы (1.4) и (1.8). Подсказка: градиент по весам слоя равен произведению градиента по его выходу и его входного сигнала.

4. Сопоставьте измеренный рост $q^{(L)}/q^{(0)}$ при $\alpha = 1$ из E2 с предсказанием (1.13) для своих глубин. Почему остаточная сеть с $\alpha = 1$ может стать неустойчивой при большой глубине, хотя у градиента есть прямой путь?
5. Найдите в E2 две конфигурации с близкой точностью на `val`, но заметно разным GI. Можно ли считать их одинаково устойчивыми к увеличению глубины? Проверьте на следующей глубине профиля.

11 Обязательные тесты

1. Монитор активаций не меняет выход модели.
2. Монитор градиентов не меняет значения градиентов.
3. Выборочная дисперсия весов после инициализации отличается от целевой не более чем на 10% при $n_{l-1}n_l \geq 10^4$.
4. Остаточный блок с нулевой ветвью возвращает свой вход с точностью 10^{-6}.
5. Сводка обучаемости ставит флаг численного сбоя при NaN или Inf.

12 Что сдать

Сдаются модули заданий 1–3, конфигурации E1–E3, файл `hypothesis.md`, тесты и отчёт. Отчёт состоит из шести частей:

1. **Профиль и протокол:** глубины, σ, оптимизатор, подобранная скорость обучения.
2. **E1:** графики по слоям, таблица $\hat{\chi}$ против χ, выводы.
3. **E2:** таблица ρ_T, GI, AVD и точности по архитектурам и глубинам; рост сигнала до обучения; найденные глубины.
4. **Карта обучаемости:** один двумерный срез, например «глубина $\times$ архитектура $\to \rho_T$», с перечнем зафиксированных факторов.
5. **E3:** гипотеза, все Δ_s, вывод.
6. **Ответы на контрфактические вопросы.**

Каждое число сопровождается ссылкой на `RUN_ID`. Графики строятся автоматически из сохранённых результатов, без ручного переноса чисел.

13 Критерии завершения и оценивания

Работа завершена, если одновременно:

- эксперименты E1–E3 выполнены, у каждого результата есть `RUN_ID`;
- тесты проходят;
- файл `hypothesis.md` зафиксирован в Git до запусков E3;
- карта обучаемости построена с перечнем зафиксированных факторов;
- на все контрфактические вопросы даны ответы с собственными числами.

Веса частей оценки те же, что в ЛР0: реализация 0,15, экспериментальный протокол 0,30, диагностика 0,30, перенос 0,25.

14 Дополнительные задания

1. **Нормализация.** Для глубокой обычной сети с неудачной инициализацией сравните варианты без нормализации, с BatchNorm и с LayerNorm. Как меняются AVD и GI? Что происходит с BatchNorm при размере batch 1 в режиме обучения?
2. **Оптимизатор.** Для конфигурации с большим GI сравните SGD с импульсом и AdamW [15]. Скорость обучения подбирайте для обоих по одной и той же сетке. Измеряйте относительное обновление слоя — отношение нормы изменения параметров за шаг к норме самих параметров. Может ли адаптивный оптимизатор скрыть плохую геометрию градиента, не устранив её причину?
3. **Якобиан.** Для сети уменьшенной ширины вычислите якобиан выхода последнего скрытого слоя по входу с помощью `torch.func.jacrev` [7]. Найдите сингулярные числа и отношение наибольшего к наименьшему. Почему одного наибольшего сингулярного числа недостаточно для вывода об обучаемости [14]?

15 Контрольные вопросы

1. Почему одинаковая дисперсия весов не даёт одинаковую дисперсию активаций для разных функций активации?
2. Какие предположения используются при выводе инициализаций Xavier и He?
3. Чем затухание активаций отличается от затухания градиентов?
4. Может ли BatchNorm компенсировать неудачную инициализацию? Устраняет ли она причину?
5. Почему остаточная связь облегчает прохождение градиента?
6. Почему масштаб остаточной ветви важнее с ростом глубины?
7. Чем обучаемость отличается от обобщающей способности?
8. Почему быстрый спад ошибки на обучении не гарантирует лучшего качества на `val`?

16 Итог

После ЛР1 вы умеете не только сказать, какая конфигурация обучилась лучше, но и объяснить почему. Инициализация и архитектура определяют, как проходит сигнал; от этого зависит размер градиентов; градиенты вместе с оптимизатором определяют ход обучения. Инструменты этой работы применимы к анализу любой глубокой модели.

Литература

[1] Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. Погружение в мир нейронных сетей. — СПб.: Питер, 2018. — 480 с.

[2] Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение / пер. с англ. А. А. Слинкина. — 2-е изд., испр. — М.: ДМК Пресс, 2018. — 652 с.

[3] PyTorch Documentation: torch.nn.init. https://docs.pytorch.org/docs/stable/nn.init.html

[4] PyTorch Documentation: torch.nn.Module (хуки `register_forward_hook`, `register_full_backward_hook`). https://docs.pytorch.org/docs/stable/generated/torch.nn.Module.html

[5] PyTorch Documentation: torch.nn.BatchNorm1d. https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm1d.html

[6] PyTorch Documentation: torch.nn.LayerNorm. https://docs.pytorch.org/docs/stable/generated/torch.nn.LayerNorm.html

[7] PyTorch Documentation: torch.func.jacrev. https://docs.pytorch.org/docs/stable/generated/torch.func.jacrev.html

[8] X. Glorot, Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. *Proc. AISTATS*, PMLR 9:249–256, 2010.

[9] K. He, X. Zhang, S. Ren, J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *Proc. ICCV*, pp. 1026–1034, 2015.

[10] S. Ioffe, C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proc. ICML*, PMLR 37:448–456, 2015.

[11] J. L. Ba, J. R. Kiros, G. E. Hinton. Layer normalization. arXiv:1607.06450, 2016.

[12] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. *Proc. CVPR*, pp. 770–778, 2016.

[13] B. Hanin, D. Rolnick. How to start training: The effect of initialization and architecture. *Advances in NeurIPS*, 2018.

[14] A. M. Saxe, J. L. McClelland, S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *Proc. ICLR*, 2014.

[15] I. Loshchilov, F. Hutter. Decoupled weight decay regularization. *Proc. ICLR*, 2019.

[16] H. Xiao, K. Rasul, R. Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. arXiv:1708.07747, 2017.